



REACHING THE STUDIO FROM SOMEWHERE ELSE

DOC NO. RL-360-REMOTE-ACCESS-A REV. A RIPOSTE LABORATORIES

The studio is a backend. It accepts uploads and runs `ffgac/ffedit` subprocesses on them, and it has no authentication of its own – on a private network it is gated by the reverse proxy in front of it. That shapes every option below.

There are two honest ways to use it remotely, and they are very different in what they cost you.

SEC.01 1. OVER A PRIVATE OVERLAY (TAILSCALE) – NOTHING NEW TO RUN

This is already working. No port is opened, no second copy of the data exists, and nothing is exposed to the internet. Verified 2026-08-14:

PIECE	STATE
Subnet route 10.0.1.0/24	advertised by x, and it holds the primary route
IP forwarding on x	<code>net.ipv4.ip_forward = 1</code>
Split DNS <code>hq.ripostelabs.xyz</code> → <code>10.0.1.99</code> (Pi-hole)	configured tailnet-wide , with the search domain set too
ACL	every device except one gets <code>*:*</code> , which covers subnet routes
TLS	the wildcard <code>*.hq.ripostelabs.xyz</code> cert already covers the studio's hostname

So from any tailnet device with `--accept-dns=true`, the studio is simply at its normal internal hostname. Nothing about the deployment changes; the Authelia gate still applies, because you are arriving through the same proxy as on the LAN.

Adding a new hostname needs no Tailscale change at all. The split-DNS route covers the whole `hq.ripostelabs.xyz` zone, so a new name resolves the moment Pi-hole and the proxy know about it.

THE ONE GAP: THE LAPTOP

The tailnet ACL deliberately restricts one device (laptop) to the mac mini only. It is the single device that **cannot** reach the studio this way, and it will not fail loudly – it will look like DNS or a dead service.

Tailscale ACLs have no deny rules, so this was implemented by enumerating every **other** device as an allowed source. Two consequences worth knowing before you debug this at 11pm:

- ▶ Any newly-added device gets zero access until it is added to the policy.
- ▶ A device removed and re-added can get a new Tailscale IP, silently breaking the host alias that grants it access.

Widening this is a deliberate policy decision, not a fix – the restriction was put there on purpose. If you do want the laptop to reach the studio, grant it that destination specifically rather than restoring blanket access.

VERIFYING IT

The pieces above were each checked directly. What was **not** verified is a full request originating from an off-LAN tailnet device, because no such device on this tailnet accepts a shell (the one that does refuses SSH, and the phone cannot be driven). If you want that proof, from a phone or laptop on the tailnet and off the LAN:

```
| https://<studio hostname>/api/health      # expect the Authelia login, then ok: true
```

A 302 to `auth.hq...` is the **correct** result and proves routing and DNS worked. It does not prove the engine is healthy – that is what `/api/health` behind the gate is for.

SEC.02

2. ON A MACHINE OF ITS OWN

Use `studio/deploy/provision.sh`. It provisions a fresh Ubuntu 22.04+ or Debian 13 host, x86-64 or arm64, and sizes the unit from that host’s own RAM and disk:

SH

```
studio/deploy/provision.sh root@<host> --ssh-key ~/.ssh/id_ed25519
```

It binds to `127.0.0.1` by default. Put a gate in front before changing that.

WHAT IT NEEDS

	MINIMUM	COMFORT- ABLE	WHY
RAM	2 GB	4 GB	one render peaks near 970 MB (~104 B/px × 9M px, measured on <code>colour.hsv</code> , the costliest op); below a ~1.4 GB ceiling the cgroup SIGKILLS the engine, which surfaces as a 502 , not an error
CPU	1 core	2-4	<code>GLITZY_MAX_WORKERS</code> is 1 by default; more workers divide the memory budget, they do not add to it
Disk	10 GB	20 GB+	cache budget (default 6 GB) + sources/exports + <code>ffglitch</code> (~110 MB)
Arch	x86-64 or arm64		<code>ffglitch</code> ships both for Linux

The script sets `GLITZY_MAX_PIXELS`, `GLITZY_MAX_WORKERS`, `GLITZY_CACHE_BUDGET` and `MemoryMax` together from what it finds. They are one setting in four places: **workers × per-render budget must stay under the cgroup cap**. Raising `GLITZY_MAX_PIXELS` alone does not let the engine survive more work – it only stops it refusing work it will then be killed for.

ON ORACLE ALWAYS FREE SPECIFICALLY

- ▶ **VM.Standard.E2.1.Micro** will not work. 1 GB of RAM against a 1.4 GB working ceiling; it will be OOM-killed doing ordinary things. The script refuses to provision below 1800 MB rather than let you discover this later.
- ▶ **VM.Standard.A1.Flex (Ampere, arm64)** is the right shape – the free allowance is 4 OCPU / 24 GB total and may be split across up to 4 instances, so a 1 OCPU / 6 GB slice is both plenty for the studio and leaves room for other tenants of the same allowance.
- ▶ ffglitch's Linux arm64 build is published as a **.7z**, not a **.zip**. It is easy to look at the download page, see **linux-x86_64.zip** and **macos-aarch64.zip**, and conclude Linux ARM does not exist. It does, and the provisioner uses it. **bsdtar** reads both formats, which is why **libarchive-tools** is installed rather than **p7zip**.
- ▶ A1 capacity in single-AD regions is the real obstacle, not the software. There is no second availability domain to fall back to, so a patient retry loop is the only lever – and a **tight** loop makes it worse, because OCI answers **429** and masks the actual capacity error.

SEC.03

WHICH ONE

If the goal is "use the studio when I'm not at home", **option 1 is already done** and costs nothing: no new machine, no second copy of the cache, no public surface, and the existing gate keeps applying.

Option 2 is for when the studio needs to serve people who are not on your tail-net – and then the authentication question stops being optional.

SOURCE studio/deploy/REMOTE-ACCESS.md SHA 0fa9d647a336 BUILT 2026-08-15